

Practica - 7 Operaciones con Cadenas de Texto (Strings)

Luis Gálvez

Tabla de contenidos

1 Proposito de la practica	1
1.1 Ejercicio 1: Limpieza de topónimos (PNN Macarena)	1
1.1.1 Desarrollo	2
1.2 Ejercicio 2: Parseo de coordenadas y WKT (Volcán Galeras) . . .	2
1.2.1 Desarrollo	3
1.3 Preguntas	4
1.3.1 Desarrollo	4

1 Proposito de la practica

1.1 Ejercicio 1: Limpieza de topónimos (PNN Macarena)

Contexto: Has recibido una base de datos antigua del IGAC con los nombres de las áreas protegidas de Colombia. Los datos fueron ingresados manualmente a lo largo de los años, por lo que tienen serios problemas de formato (espacios extra, mayúsculas inconsistentes y errores de digitación). Necesitas estandarizar el nombre del Parque Nacional Natural Serranía de la Macarena para cruzarlo con el sistema actual.

Instrucciones de código:

1. Define una variable llamada `registro_crudo` que contenga exactamente este texto: " pnn seRRania de la mACaRena "
2. Utiliza el método nativo de tu lenguaje para **eliminar los espacios en blanco** al principio y al final del texto. Guarda el resultado en `texto_sin_espacios`.
3. Convierte el texto resultante a **Formato Título** (donde la primera letra de cada palabra es mayúscula) y guárdalo en `texto_titulo`.
4. El sistema oficial requiere que las siglas "PNN" estén en mayúscula sostenida. Utiliza la función de **reemplazo** para cambiar la subcadena "Pnn" por "PNN" dentro de tu `texto_titulo`. Guarda esto como `registro_oficial`.

5. Calcula la **longitud** (cantidad de caracteres) de tu `registro_oficial`.
6. Imprime un reporte final utilizando **interpolación de cadenas** que diga exactamente: "El área protegida limpia es: [AQUI_TU_VARIABLE] y su nombre tiene [AQUI_LONGITUD] caracteres."

1.1.1 Desarrollo

```
# Registro original con errores de formato
registro_crudo = "   pnn   seRRania de la mACaRena   "

#-----
# Eliminar espacios al inicio y al final
texto_sin_espacios = registro_crudo.strip()

#-----
# Convertir a formato título
texto_titulo = texto_sin_espacios.title()

#-----
# Reemplazar "Pnn" por "PNN"
registro_oficial = texto_titulo.replace("Pnn", "PNN")

#-----
# Calcular longitud del texto final
longitud_nombre = len(registro_oficial)

#-----
# Imprimir reporte usando interpolación de cadenas
print(
    f"El área protegida limpia es: {registro_oficial} "
    f"y su nombre tiene {longitud_nombre} caracteres."
)
```

El área protegida limpia es: PNN Serrania De La Macarena y su nombre tiene 28 caracteres.

1.2 Ejercicio 2: Parseo de coordenadas y WKT (Volcán Galeras)

Contexto: Un sensor sísmico ubicado en el Volcán Galeras (Nariño) envía las alertas a través de mensajes de texto planos. Tu objetivo es extraer las coordenadas de ese mensaje de texto, convertirlas a formato numérico, y ensamblar un punto espacial estándar (Well-Known Text) para insertarlo en la base de datos PostGIS del Servicio Geológico.

Instrucciones de código:

1. Define la variable `mensaje_sensor` con el siguiente texto: "ALERTA_SISMICA:1.221,-77.359:PROFUNDIDA

2. Utiliza la función de **separación (Split)** usando los dos puntos (:) como delimitador. Extrae únicamente el bloque central que contiene las coordenadas y guárdalo en la variable `bloque_coords`.
3. Vuelve a aplicar la función de **separación (Split)**, esta vez sobre `bloque_coords`, usando la coma (,) como delimitador para separar la latitud de la longitud.
4. Las coordenadas extraídas siguen siendo texto. Conviértelas a **formato numérico decimal (Float)** y guárdalas en las variables `latitud` y `longitud`.
5. La base de datos requiere una precisión estricta. Utilizando las herramientas de formateo de tu lenguaje, construye una cadena de texto en formato WKT (`POINT(lon lat)`) asegurando que tanto la latitud como la longitud tengan **exactamente 4 decimales**. Guarda esto en `punto_wkt`. *(Nota: Recuerda que WKT usa el orden Longitud Latitud, separados por un espacio, no por coma).*
6. El sensor reportó una anomalía térmica que afectó un área de 1250000 metros cuadrados. Formatea este número agregando un **separador de miles** (coma o punto) para que sea legible en el reporte final.
7. Imprime el WKT generado y el área afectada formateada.

1.2.1 Desarrollo

```
# Mensaje recibido desde el sensor
mensaje_sensor = "ALERTA_SISMICA:1.221,-77.359:PROFUNDIDAD_5KM"

#-----
# Separar el mensaje usando ":"
partes_mensaje = mensaje_sensor.split(":")

#-----
# Extraer bloque de coordenadas
bloque_coords = partes_mensaje[1]

#-----
# Separar latitud y longitud usando ","
coords = bloque_coords.split(",")

#-----
# Convertir coordenadas a tipo float
latitud = float(coords[0])
longitud = float(coords[1])

#-----
# Construir geometría WKT con 4 decimales
# formato POINT(longitud latitud)
```

```
punto_wkt = f"POINT({longitud:.4f} {latitud:.4f})"
```

```
#-----  
# Formatear área con separador de miles  
area_afectada = 1250000  
area_formateada = f"{area_afectada:,}"
```

```
#-----  
# Imprimir resultados  
print("Geometría WKT generada:")  
print(punto_wkt)  
  
print("\nÁrea afectada:")  
print(f"{area_formateada} m²")
```

Geometría WKT generada:
POINT(-77.3590 1.2210)

Área afectada:
1,250,000 m²

1.3 Preguntas

1. **Sobre el Ejercicio 1:** ¿Por qué es una mala práctica depender únicamente de la función “Reemplazar” (`.replace()` / `gsub()`) para quitar espacios accidentales al inicio o final de un texto en bases de datos geográficas grandes, en lugar de usar funciones dedicadas como `strip()` o `trimws()`?
2. **Sobre el Ejercicio 2:** En la creación de la geometría WKT, si omitieras el paso de convertir el texto extraído a número decimal (Float) y construyeras el WKT directamente concatenando los textos originales, ¿qué impacto tendría esto al intentar realizar un cálculo de distancia espacial dentro de PostGIS o QGIS posteriormente?
3. **Pregunta General (Split en R):** Si decidiste resolver el Ejercicio 2 usando **R**, explica por qué fue necesario utilizar `unlist()` o los dobles corchetes `[[1]]` después de ejecutar la función `strsplit`. Si usaste Python o Julia, explica cómo se diferencia la salida de la función `split` en tu lenguaje respecto a R.

1.3.1 Desarrollo

1. Se necesitan funciones especializadas debido a la complejidad y diversidad de los errores que se pueden encontrar, además que funciones como `.replace()` / `gsub()` pueden generar inconvenientes alterando partes de la cadena de texto que no se deseen mutar. Funciones como `strip()`

o `trimws()` permiten editar los extremos de la cadena de texto, lo que permite eliminar espacios no deseados, saltos de línea y demás cosas que generen ruido no deseado en la cadena de texto.

2. Podrían pasarse como cadenas de texto, lo que eventualmente generaría problemas al realizar operaciones matemáticas o espaciales sobre ellos.
3. la diferencia es la forma en que devuelve el resultado, por que en python basta con el índice para acceder a la lista resultante, en R el resultado se entrega empaquetado, por lo que es preciso usar el índice 2 veces para acceder al resultado, es decir que para llamar el resultado en R se usa “`resultado[[1]][1]`”, mientras que en python o Julia se usa `resultado[0]` o `resultado[1]` respectivamente.